

GUIA DE ESTUDIO

Bases de Datos II

TRIGGERS & TRANSACTIONS EN MySQL

Teoria | Ejemplos | Ejercicios Practicos | Guia Docente

Aprende de forma progresiva con ejemplos reales y ejercicios graduados

Asignatura	Bases de Datos II
Temas	Triggers y Transactions en MySQL
Nivel	Intermedio - Avanzado
Requisitos	SQL basico, DDL/DML, Modelado relacional
Contenido	Teoria completa + 18 ejercicios practicos + Guia docente

TABLA DE CONTENIDOS

PARTE I: TRIGGERS EN MySQL

- 1.1 Que es un Trigger
- 1.2 Tipos de Triggers (BEFORE/AFTER - INSERT/UPDATE/DELETE)
- 1.3 Sintaxis y estructura
- 1.4 Referencias NEW y OLD
- 1.5 Restricciones y buenas practicas
- 1.6 Ejercicios practicos (1-8)

PARTE II: TRANSACCIONES EN MySQL

- 2.1 Que es una Transaccion
- 2.2 Propiedades ACID
- 2.3 Comandos: BEGIN, COMMIT, ROLLBACK
- 2.4 SAVEPOINT y RELEASE SAVEPOINT
- 2.5 Niveles de aislamiento
- 2.6 Bloqueos y concurrencia
- 2.7 Ejercicios practicos (9-16)

PARTE III: ESCENARIOS COMBINADOS

- 3.1 Triggers dentro de transacciones
- 3.2 Ejercicios integradores (17-18)

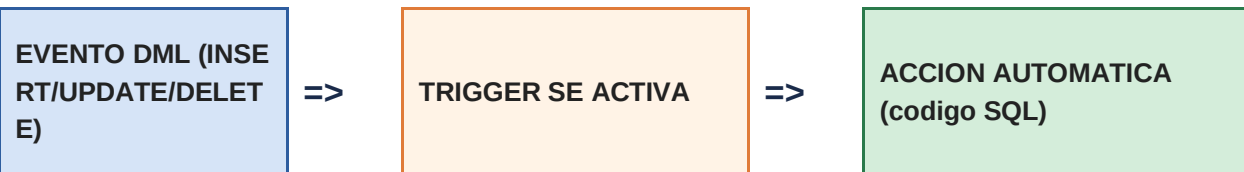
PARTE I: TRIGGERS EN MySQL

1.1 Que es un Trigger?

Un **Trigger** (disparador) es un objeto de base de datos que se asocia a una tabla y se ejecuta automaticamente cuando ocurre un evento DML especifico: **INSERT**, **UPDATE** o **DELETE**. A diferencia de los procedimientos almacenados, los triggers *no se invocan manualmente*; el motor de la base de datos los dispara de forma implicita cuando se produce el evento configurado.

Casos de uso tipicos:

- Auditar cambios en tablas criticas (historial de modificaciones).
- Mantener columnas calculadas o columnas de resumen actualizadas.
- Implementar restricciones complejas que no puede expresar una constraint simple.
- Sincronizar datos entre tablas relacionadas.
- Enviar senales a sistemas externos a traves de tablas de cola.



1.2 Tipos de Triggers

MySQL permite combinar el **momento de disparo** con el **tipo de evento**, dando lugar a 6 combinaciones posibles:

Tipo	Momento	Cuando se ejecuta	Caso de uso tipico
BEFORE INSERT	Antes	Antes de insertar la fila	Validar o modificar valores antes de guardar
AFTER INSERT	Despues	Despues de insertar la fila	Registrar en auditoria, actualizar totales
BEFORE UPDATE	Antes	Antes de actualizar la fila	Verificar reglas de negocio, formatear datos
AFTER UPDATE	Despues	Despues de actualizar la fila	Log de cambios, sincronizar tablas
BEFORE DELETE	Antes	Antes de eliminar la fila	Evitar borrados no autorizados
AFTER DELETE	Despues	Despues de eliminar la fila	Mover a tabla de archivo, limpiar dependencias

Buena practica: En MySQL >= 5.7.2, se permite definir **multiples triggers del mismo tipo** para la misma tabla, controlando el orden de ejecucion con **FOLLOWS / PRECEDES**.

1.3 Sintaxis y Estructura

La sintaxis general para crear un trigger en MySQL es:

```
DELIMITER $$  
  
CREATE TRIGGER nombre_trigger {BEFORE |  
AFTER} {INSERT | UPDATE | DELETE}  
  
ON nombre_tabla  
  
FOR EACH ROW  
  
BEGIN  
  
    -- Cuerpo del trigger: instrucciones SQL -- Se pueden usar  
    IF, CASE, variables locales, SIGNAL, etc.  
  
END$$  
  
DELIMITER ;
```

Componentes clave:

Componente	Descripcion
DELIMITER \$\$	Cambia el delimitador para que el ; interno no cierre el CREATE
nombre_trigger	Identificador unico del trigger en la base de datos
FOR EACH ROW	El trigger se ejecuta una vez por cada fila afectada (row-level)
BEGIN ... END	Bloque de codigo que puede contener multiples instrucciones
NEW	Referencia a los nuevos valores (INSERT, UPDATE)
OLD	Referencia a los valores anteriores (UPDATE, DELETE)

1.4 Referencias NEW y OLD

Dentro del cuerpo de un trigger, MySQL expone dos pseudo-registros:

Referencia	INSERT	UPDATE	DELETE
NEW.columna	Valor a insertar	Valor nuevo	No disponible (NULL)
OLD.columna	No disponible (NULL)	Valor original	Valor a eliminar

Ejemplo: Trigger BEFORE UPDATE con NEW y OLD

```
DELIMITER $$
CREATE TRIGGER trg_validar_precio
BEFORE UPDATE ON productos
FOR EACH ROW
BEGIN -- Rechazar si el nuevo precio es menor al 50% del precio
anterior IF NEW.precio < OLD.precio * 0.50 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Reduccion de precio
superior al 50% no permitida';
END IF; -- Guardar la fecha de la ultima
modificacion SET NEW.fecha_modificacion =
NOW();
END$$
DELIMITER ;
```

Buena practica: En un **BEFORE trigger** puedes modificar NEW.columna para cambiar el valor antes de que se escriba en la tabla. Util para normalizar datos, truncar strings, calcular valores derivados.

1.5 Restricciones y Buenas Practicas

- Los triggers NO pueden realizar COMMIT o ROLLBACK independiente dentro de una transaccion existente.
- No se pueden usar instrucciones de control de transacciones (COMMIT/ROLLBACK) dentro de triggers.
- Un trigger no puede invocar procedimientos que abran cursores sobre la misma tabla (recursion infinita).
- MySQL solo soporta triggers a nivel de fila (row-level), no a nivel de instruccion (statement-level).
- Los triggers se ejecutan en el mismo contexto de usuario que la instruccion que los dispara.
- Usa SIGNAL SQLSTATE para abortar la operacion y lanzar un error controlado al cliente.

Consejo: Mantén los triggers simples y enfocados en una sola responsabilidad. Triggers muy complejos dificultan el debugging. Si la logica es extensa, considera moverla a un procedimiento almacenado que el trigger invoque.

Advertencia: Evita triggers que modifiquen la misma tabla que los activa (triggers recursivos). MySQL tiene max_recursive_iterations pero es mejor evitar la recursion por diseno.

EJERCICIOS PRACTICOS - TRIGGERS (1-8)

Los siguientes ejercicios estan organizados de menor a mayor complejidad.

Esquema de tablas (parte 1 de 2)

```
CREATE TABLE clientes ( id_cliente INT AUTO_INCREMENT PRIMARY KEY,
nombre VARCHAR(100) NOT NULL, email VARCHAR(150) UNIQUE, saldo
DECIMAL(12,2) DEFAULT 0.00, activo TINYINT(1) DEFAULT 1,
fecha_registro DATETIME DEFAULT NOW() ); CREATE TABLE productos (
```

```

id_producto INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100) NOT
NULL, precio DECIMAL(10,2) NOT NULL, stock INT DEFAULT 0,
fecha_modificacion DATETIME ); CREATE TABLE ventas ( id_venta INT
AUTO_INCREMENT PRIMARY KEY, id_cliente INT NOT NULL, id_producto INT
NOT NULL, cantidad INT NOT NULL, total DECIMAL(12,2), fecha DATETIME
DEFAULT NOW(), FOREIGN KEY (id_cliente) REFERENCES
clientes(id_cliente), FOREIGN KEY (id_producto) REFERENCES
productos(id_producto)
);

```

Esquema de tablas (parte 2 de 2)

```

CREATE TABLE auditoria_precios (
id INT AUTO_INCREMENT PRIMARY KEY,
id_producto INT, precio_anterior
DECIMAL(10,2), precio_nuevo
DECIMAL(10,2), usuario
VARCHAR(100), fecha_cambio
DATETIME DEFAULT NOW() );
CREATE TABLE log_clientes_eliminados (
id INT AUTO_INCREMENT PRIMARY KEY,
id_cliente INT, nombre VARCHAR(100),
email VARCHAR(150), fecha_eliminacion
DATETIME DEFAULT NOW() );

```

Ejercicio 1: AFTER INSERT - Descuento automatico de stock

Enunciado: Cada vez que se registre una venta, el stock del producto vendido debe reducirse automaticamente en la cantidad vendida.

```

DELIMITER $$ CREATE TRIGGER
trg_descontar_stock AFTER INSERT
ON ventas
FOR EACH ROW
BEGIN
UPDATE productos
SET stock = stock - NEW.cantidad
WHERE id_producto = NEW.id_producto;
END$$
DELIMITER ;

-- Prueba: INSERT INTO ventas (id_cliente, id_producto, cantidad, total) VALUES (1, 3,
5, 250.00);

SELECT stock FROM productos WHERE id_producto = 3;

```

Ejercicio 2: BEFORE INSERT - Calcular total automaticamente

Enunciado: Al insertar una venta, el campo total debe calcularse automaticamente multiplicando cantidad por el precio del producto.

Ejercicio 3: AFTER UPDATE - Auditoria de cambio de precios

Enunciado: Cuando el precio de un producto cambie, registrar automaticamente en auditoria_precios el precio anterior, el nuevo precio, el usuario MySQL y la fecha.

Ejercicio 4: BEFORE DELETE - Log antes de eliminar cliente

Enunciado: Antes de eliminar un cliente, guardar sus datos en log_clientes_eliminados para mantener un registro historico.

Ejercicio 5: BEFORE INSERT - Validar stock disponible

Enunciado: Antes de registrar una venta, verificar que exista suficiente stock. Si el stock es insuficiente, abortar la operacion con un mensaje de error.

Consejo: SIGNAL SQLSTATE '45000' es el codigo generico de error definido por el usuario en MySQL. Siempre incluye SET MESSAGE_TEXT para dar un mensaje descriptivo.

Ejercicio 6: AFTER INSERT - Actualizar saldo del cliente

Enunciado: Cuando se registra una venta, descontar el total del saldo disponible del cliente automaticamente.

Ejercicio 7: BEFORE UPDATE - Proteger email, normalizar a minusculas

Enunciado: Antes de actualizar el email de un cliente, normalizar a minusculas y verificar que no exista otro cliente con ese email.

Ejercicio 8: AFTER DELETE - Restaurar stock al cancelar venta

Enunciado: Cuando se elimine una venta (cancelacion), devolver automaticamente la cantidad al stock del producto correspondiente.

PARTE II: TRANSACCIONES EN MySQL

2.1 Que es una Transaccion?

Una **transaccion** es un conjunto de operaciones SQL tratadas como una **unidad logica de trabajo indivisible**. O todas se completan (COMMIT), o ninguna tiene efecto (ROLLBACK). Esto garantiza la **integridad de los datos** incluso ante fallos del sistema.

Ejemplo clasico: Transferencia bancaria

```
-- Sin transaccion: si el servidor falla despues del debito, -  
- la cuenta origen pierde dinero sin que el destino lo reciba.  
  
START TRANSACTION;  
  
UPDATE cuentas SET saldo = saldo - 500 WHERE id = 1; -- Debitar  
UPDATE cuentas SET saldo = saldo + 500 WHERE id = 2; -- Acreditar  
COMMIT; -- Solo si AMBAS operaciones tuvieron exito  
  
-- Si ocurre un error entre las dos instrucciones:  
ROLLBACK; -- Se deshacen TODOS los cambios.
```

2.2 Propiedades ACID

Las propiedades ACID son los pilares que garantizan la confiabilidad de las transacciones:

Letra	Nombre	Definicion	Ejemplo
A	Atomicidad	Todas las operaciones ocurren o ninguna.	Debito y credito juntos o ninguno.
C	Consistencia	La BD pasa de un estado valido a otro valido.	El total de dinero no cambia tras una transferencia.
I	Aislamiento	Las transacciones concurrentes son independientes.	Dos transferencias simultaneas no se interfieren.
D	Durabilidad	Un COMMIT persiste aunque haya fallo del sistema.	El saldo actualizado sobrevive a un reinicio.

2.3 Comandos: START TRANSACTION, COMMIT, ROLLBACK

Comando	Efecto	Notas
START TRANSACTION (o BEGIN)	Inicia una nueva transaccion explicita.	Desactiva el autocommit para esa sesion.
COMMIT	Confirma todos los cambios.	Los cambios son permanentes y visibles.

ROLLBACK	Deshace todos los cambios desde START TRANSACTION.	Los datos quedan como antes de la transaccion.
SET autocommit = 0	Deshabilita autocommit a nivel de sesion.	Cada DML queda pendiente hasta el COMMIT.
SET autocommit = 1	Habilita autocommit (comportamiento por defecto).	Cada DML se confirma inmediatamente.

Patron tipico con manejo de errores:

```

DELIMITER $$

CREATE PROCEDURE transferir_saldo( IN p_origen INT, IN
p_destino INT, IN p_monto DECIMAL(12,2))
BEGIN DECLARE EXIT HANDLER FOR
SQLEXCEPTION
BEGIN
ROLLBACK; RESIGNAL; -- Re-lanza el error
al cliente
END;

START TRANSACTION;
UPDATE cuentas SET saldo = saldo - p_monto WHERE id = p_origen;
UPDATE cuentas SET saldo = saldo + p_monto WHERE id = p_destino;

IF (SELECT saldo FROM cuentas WHERE id = p_origen) < 0 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Saldo insuficiente';
END IF;
COMMIT;
END$$

DELIMITER ;

CALL transferir_saldo(1, 2, 300.00);

```

2.4 SAVEPOINT y RELEASE SAVEPOINT

Los **SAVEPOINTS** son marcadores intermedios dentro de una transaccion. Permiten deshacer parte del trabajo sin cancelar todo. Son utiles en transacciones largas y complejas.

```

START TRANSACTION;

INSERT INTO pedidos (id_cliente, total) VALUES (1, 500.00);

SAVEPOINT sp_pedido_creado; -- Punto de guardado 1

INSERT INTO detalle_pedido (id_pedido, id_producto, cantidad)
VALUES (LAST_INSERT_ID(), 5, 2);

SAVEPOINT sp_detalle_ok; -- Punto de guardado 2

-- Si algo falla, podemos volver solo al sp_pedido_creado: ROLLBACK TO SAVEPOINT
sp_pedido_creado;

-- O confirmar todo:
COMMIT;

-- Liberar un savepoint (opcional):
RELEASE SAVEPOINT sp_detalle_ok;

```

Consejo: ROLLBACK TO SAVEPOINT no finaliza la transaccion. Simplemente deshace los cambios hasta ese punto. Debes emitir COMMIT o ROLLBACK completo para terminarla.

2.5 Niveles de Aislamiento (Isolation Levels)

El nivel de aislamiento define cuanto ve una transaccion de los cambios de otras transacciones concurrentes. MySQL/InnoDB soporta los 4 niveles del estandar SQL:

Nivel	Dirty Read	Non-Repeatable	Phantom Read	Uso recomendado
READ UNCOMMITTED	SI	SI	SI	Muy raro en produccion.
READ COMMITTED	NO	SI	SI	Reportes con variacion tolerable.
REPEATABLE READ (DEFAULT MySQL)	NO	NO	SI*	Transacciones OLTP tipicas.
SERIALIZABLE	NO	NO	NO	Max consistencia, alto costo.

** InnoDB usa MVCC para eliminar phantom reads en REPEATABLE READ en la mayoria de casos.*

Fenomenos de concurrencia:

- **Dirty Read:** Una transaccion lee datos que otra modifico pero aun no confirmo.
- **Non-Repeatable Read:** Una transaccion lee la misma fila dos veces y obtiene valores diferentes.
- **Phantom Read:** Una transaccion ejecuta la misma consulta dos veces y obtiene un conjunto diferente de filas.

```
-- Consultar el nivel de aislamiento actual:  
SELECT @@transaction_isolation;
```

```
-- Cambiar para la sesion actual: SET SESSION  
TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
  
-- Cambiar para la siguiente transaccion:  
SET TRANSACTION ISOLATION LEVEL  
SERIALIZABLE;  
START TRANSACTION;  
  
-- operaciones...  
COMMIT;
```

2.6 Bloqueos y Concurrencia

InnoDB usa bloqueos a nivel de fila para maximizar la concurrencia. Tipos principales:

- **Shared Lock (S):** Permite que multiples transacciones lean la misma fila. Se adquiere con SELECT ... FOR SHARE.

- **Exclusive Lock (X):** Solo una transaccion puede tener este bloqueo. Se adquiere automaticamente en INSERT/UPDATE/DELETE, o con SELECT ... FOR UPDATE.

```
-- Bloqueo compartido: otras sesiones pueden leer pero no modificar
START TRANSACTION;
SELECT * FROM cuentas WHERE id = 1 FOR SHARE;

-- Bloqueo exclusivo: ninguna otra sesion puede leer ni modificar
START TRANSACTION;
SELECT * FROM cuentas WHERE id = 1 FOR UPDATE;
UPDATE cuentas SET saldo = saldo - 100 WHERE id =
1;
COMMIT;
```

Advertencia: El DEADLOCK ocurre cuando dos transacciones esperan mutuamente el bloqueo que la otra tiene. MySQL lo detecta y aborta una transaccion. Para evitarlos: adquiere bloqueos siempre en el mismo orden y mantén las transacciones cortas.

EJERCICIOS PRACTICOS - TRANSACTIONS (9-16)

Se agrega la tabla de cuentas al esquema anterior:

```
CREATE TABLE cuentas ( id INT AUTO_INCREMENT PRIMARY
KEY, id_cliente INT NOT NULL, tipo
ENUM('corriente','ahorro') DEFAULT 'ahorro', saldo
DECIMAL(12,2) DEFAULT 0.00, FOREIGN KEY (id_cliente)
REFERENCES clientes(id_cliente)
); CREATE TABLE pedidos ( id INT
AUTO_INCREMENT PRIMARY KEY,
id_cliente INT, total DECIMAL(12,2),
estado VARCHAR(20) DEFAULT
'pendiente'
);
```

Ejercicio 9: Transaccion simple con COMMIT y ROLLBACK

Enunciado: Insertar un nuevo cliente y registrar su primera compra. Si ocurre cualquier error, deshacer todo.

```
START TRANSACTION;
INSERT INTO clientes (nombre, email, saldo)
VALUES ('Ana Torres', 'ana@email.com', 1000.00);
SET @id = LAST_INSERT_ID(); INSERT INTO ventas (id_cliente,
id_producto, cantidad, total)
VALUES (@id, 1, 2, 150.00);
COMMIT; -- Si todo OK
-- ROLLBACK; -- Si algo falla
```

Ejercicio 10: Transferencia bancaria con validacion

Enunciado: Implementar una transferencia entre dos cuentas. Validar que el saldo sea suficiente.

Ejercicio 11: Uso de SAVEPOINT en proceso de pedido

Enunciado: Registrar un pedido con multiples pasos. Usar SAVEPOINTS para poder deshacer pasos individuales.

Ejercicio 12: Autocommit OFF - Insercion de lote de productos

Enunciado: Insertar un lote de 5 productos en una sola transaccion usando autocommit deshabilitado.

Ejercicio 13: READ COMMITTED - Demostrar Non-Repeatable Read

Enunciado: Abrir dos sesiones. En la Sesion A, leer el saldo dos veces. Entre lecturas, la Sesion B modifica el saldo. Observar el comportamiento.

Ejercicio 14: FOR UPDATE - Reserva de asientos concurrente

Enunciado: Simular un sistema de reserva donde dos usuarios intentan reservar el ultimo asiento disponible simultaneamente.

Ejercicio 15: DECLARE HANDLER - Captura de errores con ROLLBACK

Enunciado: Crear un procedimiento que intente insertar en una tabla con constraint de clave duplicada, capturando el error con HANDLER.

Ejercicio 16: Proceso de compra completo - multiples tablas

Enunciado: Implementar el proceso completo de una compra: verificar stock, registrar venta, actualizar stock y debitar saldo del cliente.

PARTE III: ESCENARIOS COMBINADOS

3.1 Triggers dentro de Transacciones

Los triggers se ejecutan en el **mismo contexto transaccional** que la instruccion DML que los dispara:

- Si la transaccion hace ROLLBACK, las operaciones realizadas por el trigger tambien se deshacen.
- Si el trigger falla (ej: SIGNAL), toda la transaccion se deshace automaticamente.
- Los triggers no pueden emitir COMMIT o ROLLBACK propios; participan en la transaccion del llamador.
- Combinar triggers y transacciones permite crear sistemas de auditoria robustos.

Ejercicio 17: INTEGRADOR - Trigger + Transaccion: Auditoria atomica

Enunciado: Al realizar una transferencia dentro de una transaccion, un trigger debe registrar cada movimiento en historial. Si la transaccion falla, el historial tampoco se registra.

```
CREATE TABLE historial_movimientos
( id INT AUTO_INCREMENT PRIMARY
KEY, id_cuenta INT, tipo
ENUM('debito','credito'), monto
DECIMAL(12,2), saldo_ant
DECIMAL(12,2), saldo_nuevo
DECIMAL(12,2), fecha DATETIME
DEFAULT NOW() );

DELIMITER $$ CREATE TRIGGER
trg_historial_cuenta AFTER UPDATE
ON cuentas
FOR EACH ROW
BEGIN
IF NEW.saldo < OLD.saldo THEN
INSERT INTO historial_movimientos
(id_cuenta, tipo, monto, saldo_ant, saldo_nuevo) VALUES (NEW.id,
'debito', OLD.saldo - NEW.saldo, OLD.saldo, NEW.saldo);
ELSE
INSERT INTO historial_movimientos
(id_cuenta, tipo, monto, saldo_ant, saldo_nuevo) VALUES (NEW.id,
'credito', NEW.saldo - OLD.saldo, OLD.saldo, NEW.saldo);
END IF;
END$$

DELIMITER ;

-- El trigger se dispara automaticamente dentro de la transaccion.
-- Si hay ROLLBACK, el historial tambien se deshace.
CALL sp_transferir(1, 2, 100.00);
SELECT * FROM historial_movimientos;
```

Ejercicio 18: INTEGRADOR FINAL - Sistema de inventario completo

Enunciado: Combinar: (a) trigger BEFORE INSERT para validar stock y calcular total, (b) trigger AFTER INSERT para actualizar inventario, (c) todo dentro de una transaccion.

```
-- TRIGGER 1: Validacion antes de la venta
DELIMITER $$
CREATE TRIGGER trg_venta_before
BEFORE INSERT ON ventas
FOR EACH ROW
BEGIN
    DECLARE v_stock INT; DECLARE v_precio DECIMAL(10,2);
    SELECT stock, precio INTO v_stock, v_precio FROM
    productos WHERE id_producto = NEW.id_producto;
    IF v_stock < NEW.cantidad THEN SIGNAL SQLSTATE '45000' SET
    MESSAGE_TEXT = 'Sin stock suficiente';
    END IF; SET NEW.total = NEW.cantidad *
    v_precio;
END$$

-- TRIGGER 2: Acciones despues de la venta
CREATE TRIGGER trg_venta_after
AFTER INSERT ON ventas
FOR EACH ROW
BEGIN UPDATE productos SET stock = stock -
NEW.cantidad
WHERE id_producto = NEW.id_producto; UPDATE
clientes SET saldo = saldo - NEW.total WHERE
id_cliente = NEW.id_cliente;
END$$
DELIMITER ;

-- Usar con transaccion:
START TRANSACTION; INSERT INTO ventas (id_cliente, id_producto,
cantidad) VALUES (1, 2, 5); -- Los triggers se disparan
automaticamente.
COMMIT;
```